

Literature Review: Sorting Boxes by Colour with a UR10e Robot

Team: [team 06] · Members: [Youssef hazem – ID:58-2769], [imaged emil – ID:58-3830], [tony hany – ID:58-6506], [anas mohamed – ID:58-2897],[ahmed adham –ID:58-3529]

1. What we are building

We are building a robot cell that **sorts boxes by colour**. Three boxes (red, green and blue) wait on a conveyor in front of a UR10e robot arm. The robot picks up each box with a suction cup and drops it into the bin of the same colour. The bins are on the robot's left, right and behind it. To keep the project simple, the conveyor does not move, the box positions are read directly from the simulator (no camera), and the suction cup is simulated by attaching the box to the robot's tool while it is carried.

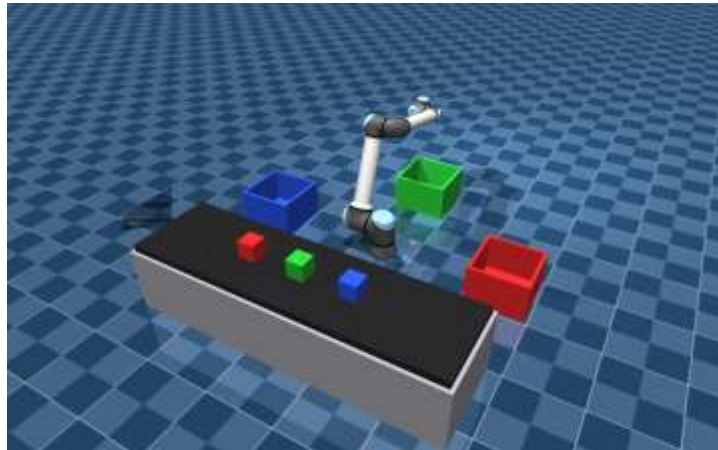


Fig. 1 – Draft cell in MuJoCo: UR10e robot, conveyor with three boxes, and three coloured bins.

2. How this problem is normally solved

Kinematics. The standard way to model a robot arm is the Denavit–Hartenberg (DH) convention: a frame is placed on each joint, and four parameters per link give the transform from one frame to the next. Multiplying these transforms gives the forward kinematics (joint angles $\rightarrow$ tool position) [1]. Inverse kinematics (tool position $\rightarrow$ joint angles) is solved either analytically with closed-form equations, which exist for the UR10e [2], or numerically by iterating with the Jacobian. The Jacobian also links joint velocities to tool velocities [3], [4].

Motion planning. Long moves are usually planned in joint space with cubic or quintic polynomials, so each joint starts and stops smoothly. Short moves where the path matters, like going straight down to a part, are planned as a straight line in task space and converted to joint angles with inverse kinematics [3], [4].

Control. The most common controller is a separate PD or PID loop on each joint. It is simple and works well at moderate speeds. Computed-torque control uses the robot's dynamic model to cancel gravity and inertia, so it follows the planned path more accurately [3], [4].

Simulation. The system is normally tested in a simulator before a real robot is used. Common choices are Gazebo [5], PyBullet [6] and MuJoCo [7]; MuJoCo is fast and handles contact between objects well. ROS 2 is the standard software layer that connects the control code to the robot or simulator by sending messages on named topics [8].

3. The robot we picked and why

We picked the **Universal Robots UR10e**, a collaborative robot (made to work safely near people [9]) with 6 joints, a 12.5 kg payload and a 1.3 m reach [10]. Why this robot:

1. **6 DoF**, which is inside the required 4–7 DoF range.
2. **Published DH parameters [10]**, so we can check our own DH table.
3. **A known closed-form inverse kinematics [2]**, so we can check our IK results.
4. **A ready-made MuJoCo model** from MuJoCo Menagerie [11], so we do not have to build the model ourselves.
5. **Enough reach:** we checked in simulation that it can reach every box and every bin in our cell.

4. Draft project flow

One sorting cycle (repeated until the conveyor is empty):

1. **Home:** wait in a safe pose and read the position and colour of the next box.
2. **Approach:** move to 10 cm above the box, then go straight down to it.
3. **Grasp and lift:** turn on the suction and lift the box straight up.
4. **Move:** swing smoothly to a point above the bin with the same colour.
5. **Place and return:** lower the box, turn off the suction, and go back home.

Plan: M02 – frames, DH table, forward and inverse kinematics, joint sliders in MuJoCo, start of the GUI. M03 – velocity and acceleration kinematics (Jacobian), checked against the simulator, and the full factory environment. M04 – joint-space and task-space trajectories, joint control, and the complete sorting cycle.

References

- [1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Pearson, 2018.
- [2] K. P. Hawkins, "Analytic inverse kinematics for the Universal Robots UR-5/UR-10 arms," Technical report, Georgia Institute of Technology, 2013.
- [3] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
- [4] M. W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, 2nd ed., Wiley, 2020.
- [5] N. Koenig, A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," *Proc. IEEE/RSJ IROS*, pp. 2149–2154, 2004.
- [6] E. Coumans, Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. <https://pybullet.org>
- [7] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *Proc. IEEE/RSJ IROS*, pp. 5026–5033, 2012.
- [8] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.
- [9] V. Villani, F. Pini, F. Leali, C. Secchi, "Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications," *Mechatronics*, vol. 55, pp. 248–266, 2018.
- [10] Universal Robots, "UR10e technical specifications" and "DH parameters for calculations of kinematics and dynamics," universal-robots.com.
- [11] K. Zakka, Y. Tassa, and MuJoCo Menagerie contributors, "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," 2022. https://github.com/google-deepmind/mujoco_menagerie